

SESSION 3 · MEMORY, RETRIEVAL AND RAG

Giving Agents a Brain

Retrieval, RAG and memory: how an agent finds what it never read and remembers what it was told.

AI, ML, and Financial Markets Workshop Series · Gies MSBA · Fri, Oct 2, 2026



Can you relate?

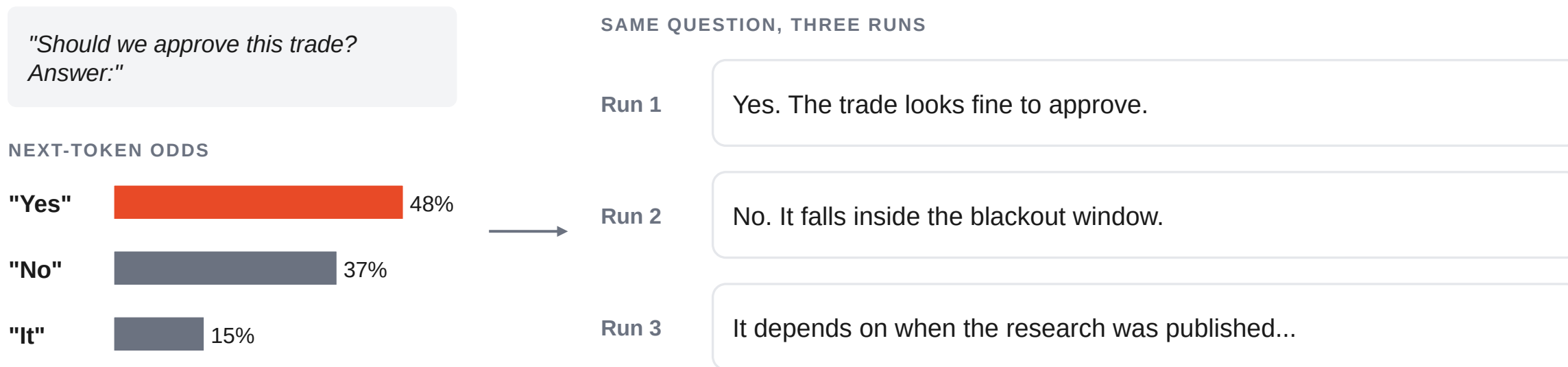
THE GOAL OF THE SERIES

We're building an investment research agent, one capability a week



Every week: the agent recommends, a person decides.

An LLM writes one token at a time, rolling weighted dice each step



Illustrative odds

80

different answers from 1,000 runs of the same prompt, even with temperature set to 0 ("no randomness").

Thinking Machines Lab, Defeating Nondeterminism in LLM Inference (Sep 2025). Qwen3-235B, one prompt.

Jev doesn't write an answer. It fills in a form you define

LLM · writes text

"I'd probably lean towards declining, though it may be worth escalating..."

- ✗ Code has to parse the sentence
- ✗ Can invent an option you never offered
- ✗ No reliable sense of how sure it is

Jev · fills a form

approve		0.04
decline		0.83
escalate		0.13

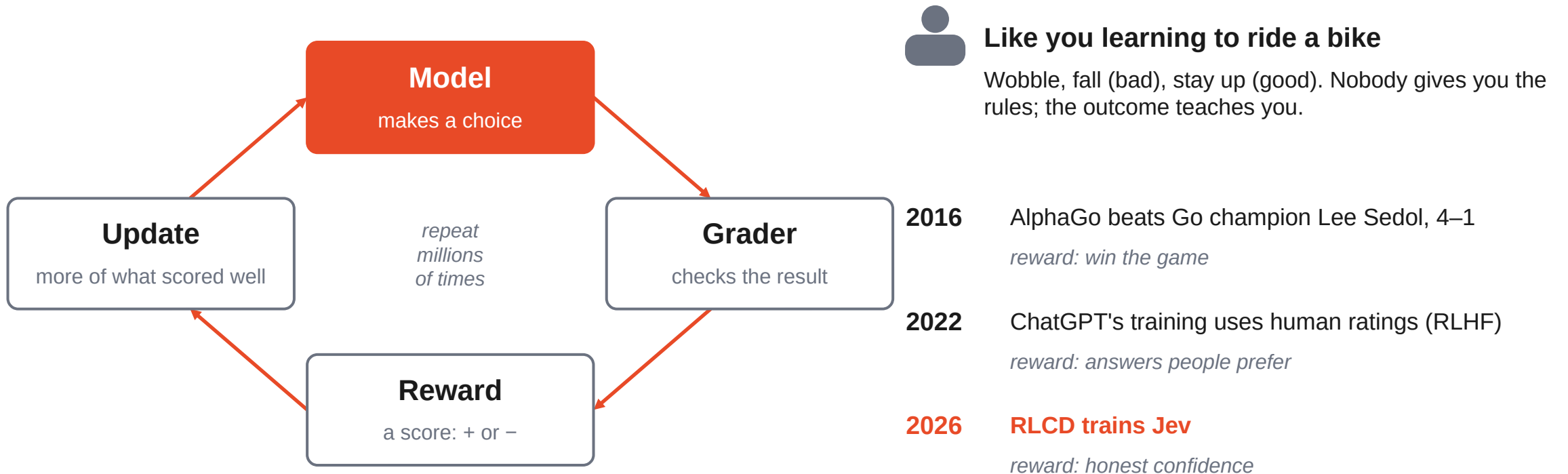
- ✓ Always one of your options, in your format
- ✓ Cannot invent a fourth option
- ✓ Confidence comes with every answer

Why it's a big deal: software can act on the answer directly. Vercel, Cloudflare, LangChain and Langfuse added Jev within three days.

Probabilities illustrative. Adoption: Forbes, Sep 19, 2026. Product claims: TypeSafe AI.

FIRST: WHAT IS REINFORCEMENT LEARNING?

Reinforcement learning: try, get scored, do more of what works

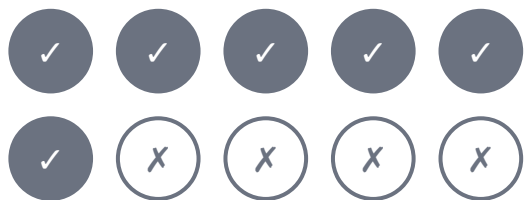


The reward decides what the model becomes. Change the reward, change the behaviour.

Calibrated: when it says 80% sure, it's right about 80% of the time

Overconfident model

10 answers it called "95% sure"



6 of 10 right

Says 95%, is right 60%. You can't trust its confidence.

Calibrated model

10 answers it called "80% sure"



8 of 10 right

Says 80%, is right 80%. Now a threshold means something.

Why it matters: "act above 0.9, send the rest to a person" only works if 0.9 really means 90%.

Illustrative counts. Definition from TypeSafe's docs: outcomes given 0.8 should happen about 80% of the time.

The idea: reward honest confidence, punish confident mistakes

A SCORING RULE AS THE REWARD (Brier score: penalty = (outcome – confidence)²)

Model's answer	Turns out	Penalty
Says 90% sure	right	0.01
Says 90% sure	wrong	0.81
Says 60% sure	right	0.16
Says 60% sure	wrong	0.36

Over many answers, the lowest total penalty comes from saying exactly how often you're right. Bluffing costs more.

TYPESAFE HAS SAID

- ✓ RL training aimed at calibration
- ✓ Trained partly on synthetic data
- ✓ Outputs: a choice, score or yes/no + probability

NOT PUBLISHED YET

- ✗ The actual reward formula
- ✗ Model size, training data size
- ✗ Calibration measurements

The table shows the textbook idea behind calibration training, not TypeSafe's published recipe.

Use it where the same small decision is made thousands of times

Trade pre-clearance

CHOICE

"Approve this employee trade?"

approve | decline | hold

Low confidence → compliance officer

Support ticket routing

CHOICE

"Which team gets this ticket?"

billing | tech | account | sales

Low confidence → triage queue

Fraud flag

YES / NO

"Is this card payment suspicious?"

yes | no

Low confidence → analyst review

Earnings-call tone

SCORE

"How positive is this guidance?"

1 2 3 4 5

Low confidence → analyst reads it

Product matching

YES / NO

"Are these two listings the same item?"

yes | no

Low confidence → manual check

Agent's next step

CHOICE

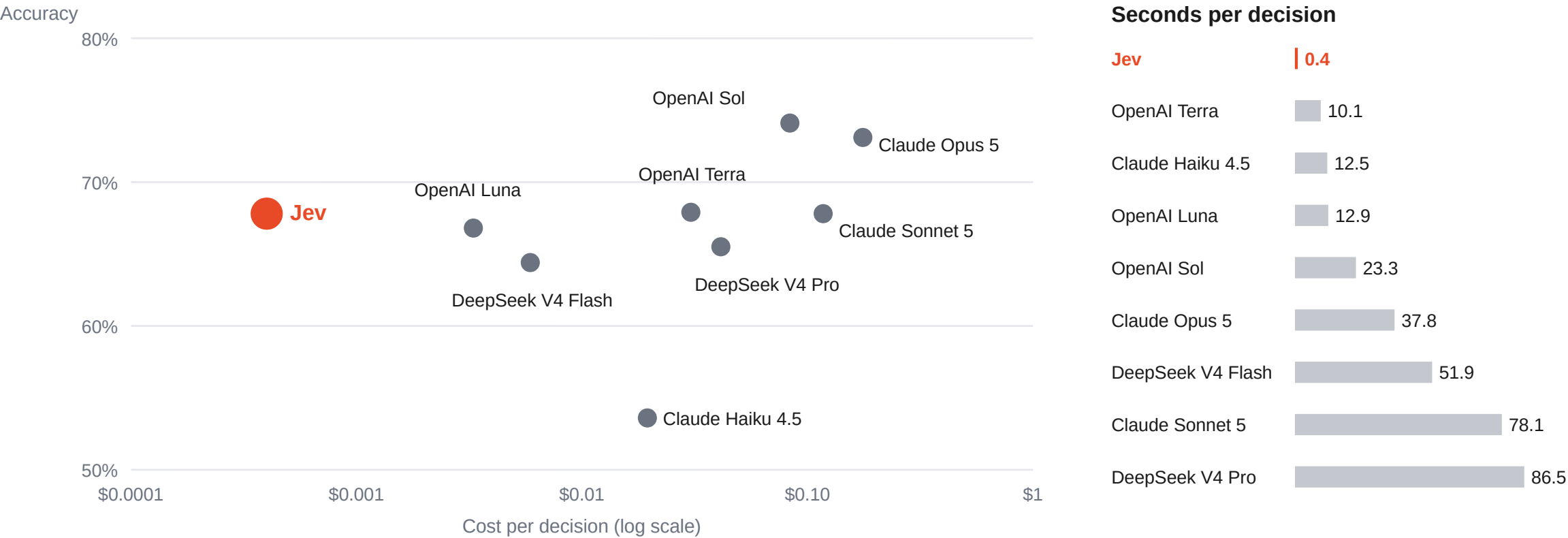
"Which tool should the agent call?"

search | financials | ask user

Low confidence → ask the user

Reported: one developer ran 9,081 product-matching decisions for \$0.32 in 13 minutes (Forbes).

Jev matches mid-tier LLM accuracy for a fraction of a cent



Same accuracy as OpenAI Terra: 76× cheaper, 25× faster. Top models still lead by about 6 points.

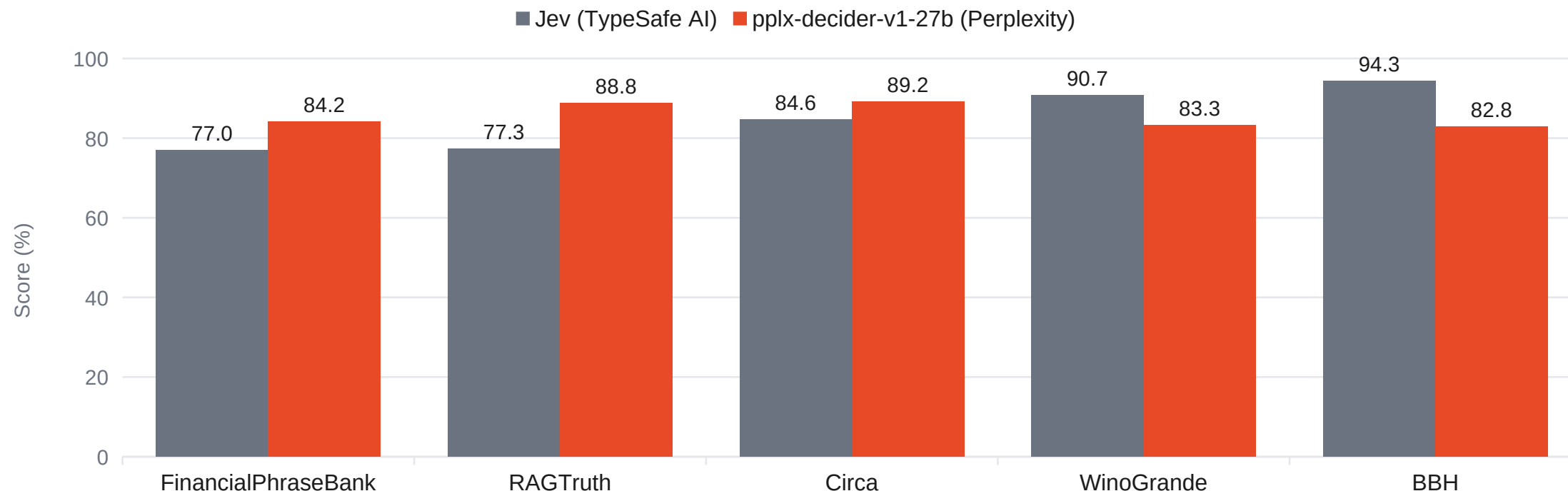
TypeSafe workflow evals (evals.typesafe.ai), 4-workflow average. Graded against GPT-6 Astra + Claude Fable 5.1 answers. Vendor-run.

Jev started the category; Cloudflare and Perplexity shipped open rivals two weeks later

	Jev · TypeSafe AI	Clef · Cloudflare	Decider · Perplexity
Released	Sep 15, 2026	Oct 1, 2026	Oct 1, 2026
Weights	Closed	Open (Apache 2.0)	Open (Apache 2.0)
Speed	70–500 ms	209 ms; Flash 39 ms	Under 2 s, short inputs
Price / M input	\$0.042, output free	Not disclosed	\$0.04, output free

All three return a choice, score or yes/no with a confidence. Figures are vendor-reported.

Perplexity's open model edges Jev overall, but each wins different tests



Overall: Decider 85.7%, Jev 84.5%. A one-point gap, chosen by the challenger.

Perplexity, pplx-decider-v1-27b model card (Hugging Face), Oct 2026. Vendor-reported.

Three new frontier models in eight days, each cheaper to run

SEP 22, 2026

Claude Opus 5.5

Anthropic · top model

\$4 in / \$20 out per M tokens

- Priced 20% below Opus 5
- ~40% cheaper to run on typical work
- Close to Fable 5.1 on most work

SEP 28, 2026

Claude Sonnet 5.5

Anthropic · everyday model

\$2 in / \$10 out per M tokens

- Same price as Sonnet 5
- 30%+ faster than Sonnet 5
- Built for clear-scope everyday tasks

SEP 29, 2026

GPT-6.1 Sol

OpenAI · launched at DevDay

\$2 in / \$10 out per M tokens

- About 1/5 of GPT-6 Astra's price
- Near-Astra on coding and computer use
- Cached input 95% off: \$0.10

INPUT PRICE, \$ PER MILLION TOKENS



Today: three ways to put the right thing on the desk

01 Retrieval and RAG

Find the handbook clause for any question

02 Short-term memory

Keep a long conversation on a small desk

03 Long-term memory

Keep what matters after the session ends

04 Hands-on

Two Colab notebooks on Lumen

Open both notebooks; both run on Lumen



3.1 Retrieval and RAG



3.2-3.3 Agent memory

ONE-TIME SETUP

- 1 Sign in to Lumen, create an API key.
- 2 Add it to Colab Secrets as LUMEN_API_KEY.
- 3 Run the first cell. See the Setup page if stuck.

01

Retrieval and RAG

A model never read the firm's handbook. Find the clause and put it on the desk.

PART 1 · RETRIEVAL AND RAG

Background

THE PROBLEM

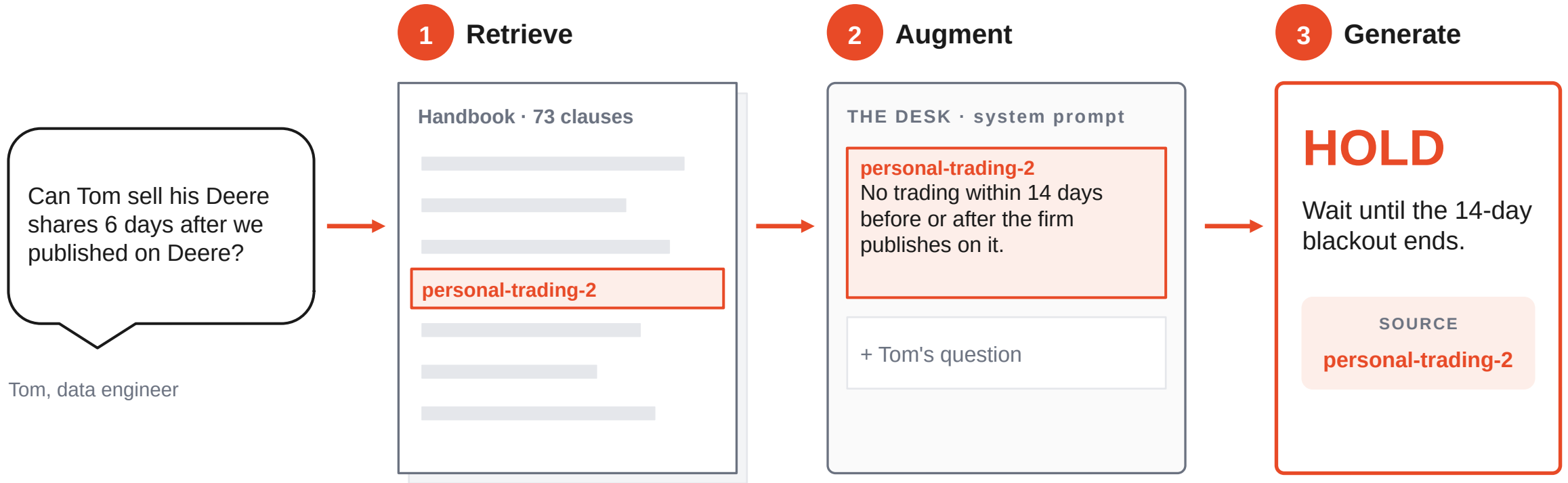
Asked about the firm's rules, a model answers confidently from general knowledge

- Tom holds Deere. We published on Deere 6 days ago. Can he sell?
- Model: generally, employees may sell shares they own.
- Firm rule: 14-day blackout. The model never read it.

PART 1 · RETRIEVAL AND RAG

RAG

RAG: find the passage, put it on the desk, answer from it



The model no longer has to remember the rule. Only read it.

Search always found the documents. LLMs now read them for you

Pre-AI

search engines, 1990s–2010s

Query

"blackout period trading rule"

Keyword index

BM25 matches the exact words

10 links

A ranked list of documents

A person reads

Opens, skims, writes the answer

Post-AI

RAG, 2020 onward

Question

"Can I sell 6 days after we published?"

Hybrid search

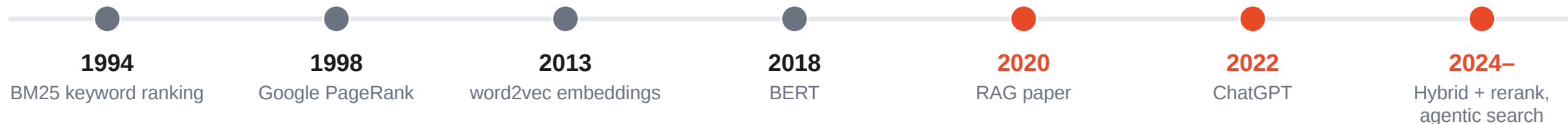
Keywords + embeddings (meaning)

Top passages

Put on the model's desk

The LLM reads

Writes the answer + cites the source



The retrieval half is decades old. What changed is who reads the results.

You already use RAG most days, usually without noticing

Google AI Overviews

"how long to boil an egg"

RETRIEVES top web pages

YOU GET summary + source links

Perplexity / ChatGPT search

"what did the Fed do this week?"

RETRIEVES live news articles

YOU GET answer with numbered citations

Gmail / Outlook assistants

"when does my flight leave?"

RETRIEVES your own emails

YOU GET the time, from the booking email

Chat with a PDF

"what's the cancellation fee?"

RETRIEVES pages of the PDF you opened

YOU GET answer + the page it's on

Customer support bots

"how do I reset my card PIN?"

RETRIEVES the company's help articles

YOU GET steps + a link to the article

Amazon's Rufus

"is this jacket waterproof?"

RETRIEVES listing, Q&A and reviews

YOU GET answer about that product

Same three steps every time: find the passages, put them on the desk, answer from them.

PART 1 · RETRIEVAL AND RAG

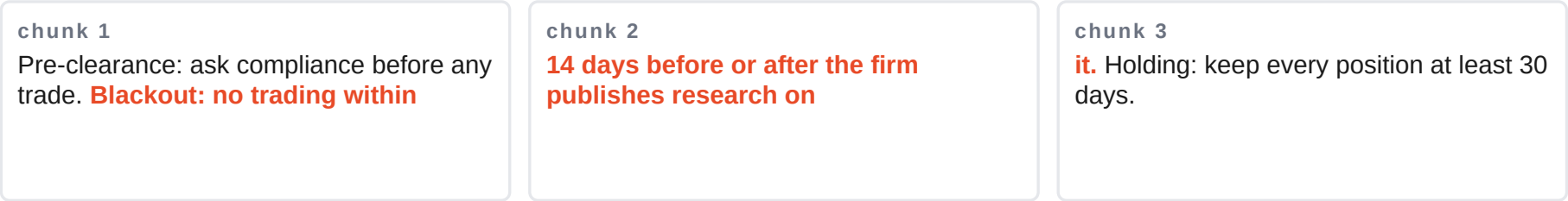
Chunking

Where you cut decides whether the rule survives whole

Fixed size

10 words, no overlap

X rule split



With overlap

16 words, repeat 10

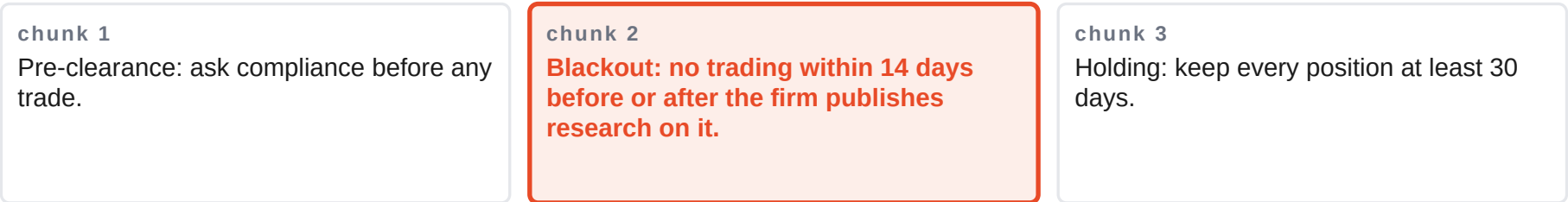
✓ rule whole



By structure

one clause per chunk

✓ rule whole



Orange = the blackout rule. Handbook: cut by structure. A 10-K: fixed size with overlap.

CHUNK SIZE

Too small cuts the rule in half; too large buries it

Too small



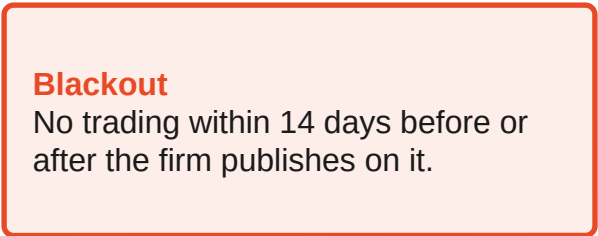
Neither piece matches

Too large



Matches everything weakly

Right size



One rule per chunk

There's no magic number. Test a few sizes on real questions

1 Start from structure

If the document has clauses, sections or headings, cut there first

2 Pick 3 sizes to try

e.g. ~200, ~400, ~800 tokens

3 Write 20–30 real questions

For each, mark the passage that answers it

4 Measure and pick

Is the right passage in the top results? How much junk comes with it?

RULES OF THUMB

Short, factual questions

→ smaller chunks

Questions needing context

→ larger chunks

One idea per chunk

→ the best target

Never over the limit

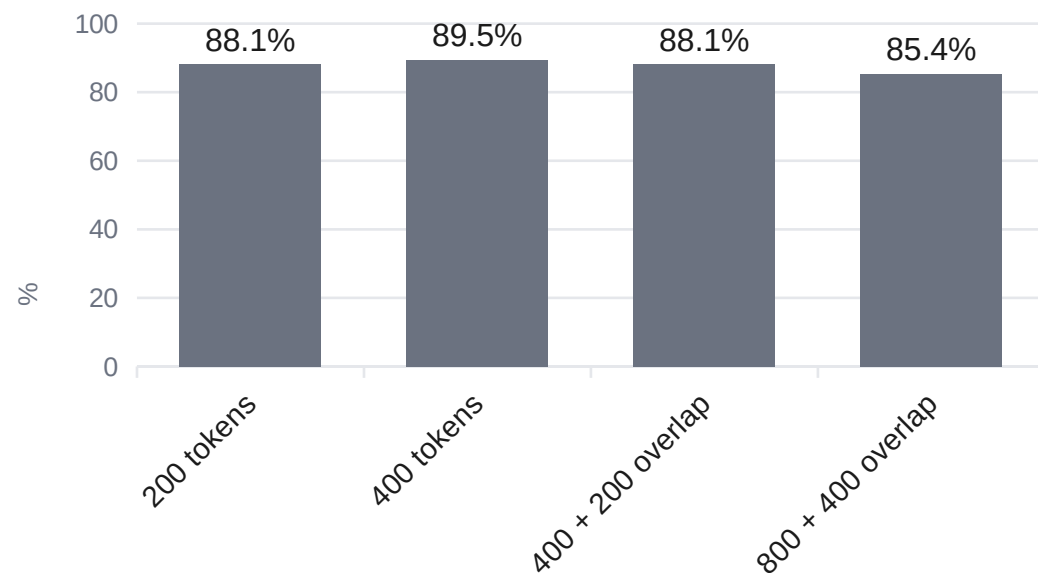
of your embedding model

Pick the smallest size that still finds the right passage.

Bigger chunks barely change recall, but precision falls from 7% to 1.5%

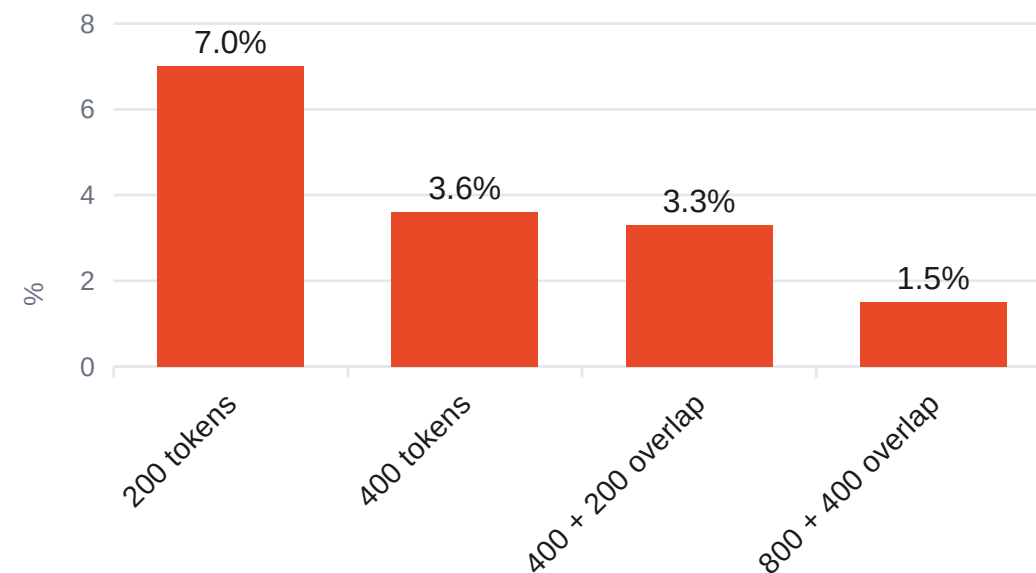
Recall

share of the answer text that was retrieved



Precision

share of retrieved text that was relevant



200-token chunks: about the same recall, 4.7× the precision of 800-token chunks.

Chroma, Evaluating Chunking Strategies for Retrieval (Jul 2024). Recursive splitter, text-embedding-3-large.

CHECKPOINT

You split a long policy document into small fixed-size chunks, with no overlap.

A question about one rule no longer finds it. Most likely reason?

- A The chunks are too long for the model to read
- B The rule was split across two chunks, and neither half matches
- C Chunks without overlap cannot be searched

CHECKPOINT · ANSWER

You split a long policy document into small fixed-size chunks, with no overlap.

A question about one rule no longer finds it. Most likely reason?

- A The chunks are too long for the model to read
- B The rule was split across two chunks, and neither half matches**
- C Chunks without overlap cannot be searched

Neither half of a split rule says enough to match. Overlap or structure fixes it.

PART 1 · RETRIEVAL AND RAG

Embeddings

TERM

Embedding

A list of numbers that stands for a piece of text. Similar meaning, similar numbers.

WHY IT MATTERS

"Quiet period after we publish" finds the blackout rule with no words in common.

Similar meaning gets a similar pattern of numbers

"No employee may trade within 14 days after the firm publishes." →



handbook clause

"Can I trade during the quiet period after we publish?" →



question

"Meals are reimbursed up to \$60." →

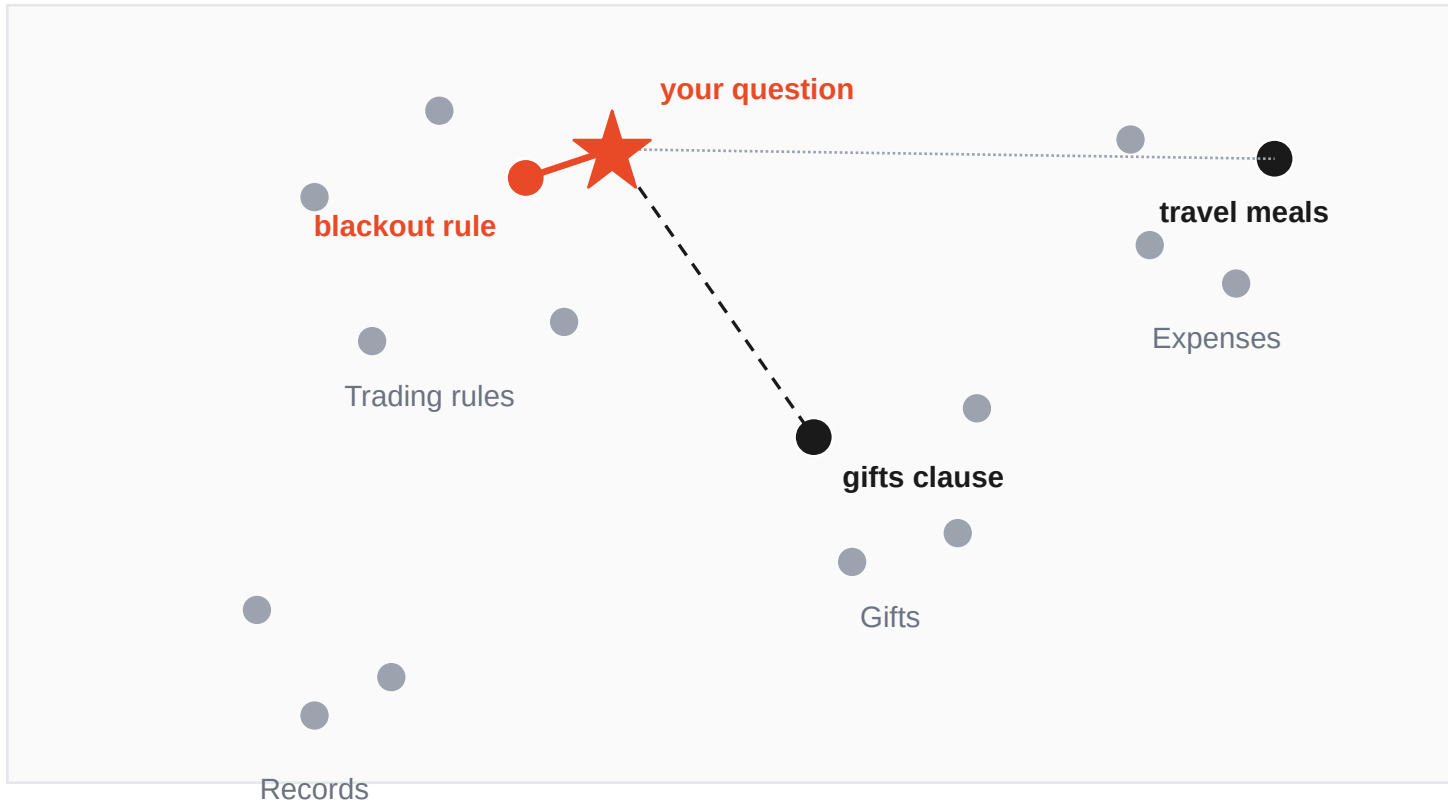


unrelated clause

Rows 1 and 2 share almost no words, but their patterns nearly match.

Illustrative: 24 of 256 numbers shown, values drawn for teaching, not real model output.

Search by meaning = find the clauses nearest the question



★ "Can I trade during the quiet period after we publish?"

- 1 **Blackout rule**
closest: no shared words
- 2 **Gifts clause**
also mentions the blackout
- **Travel meals**
far away

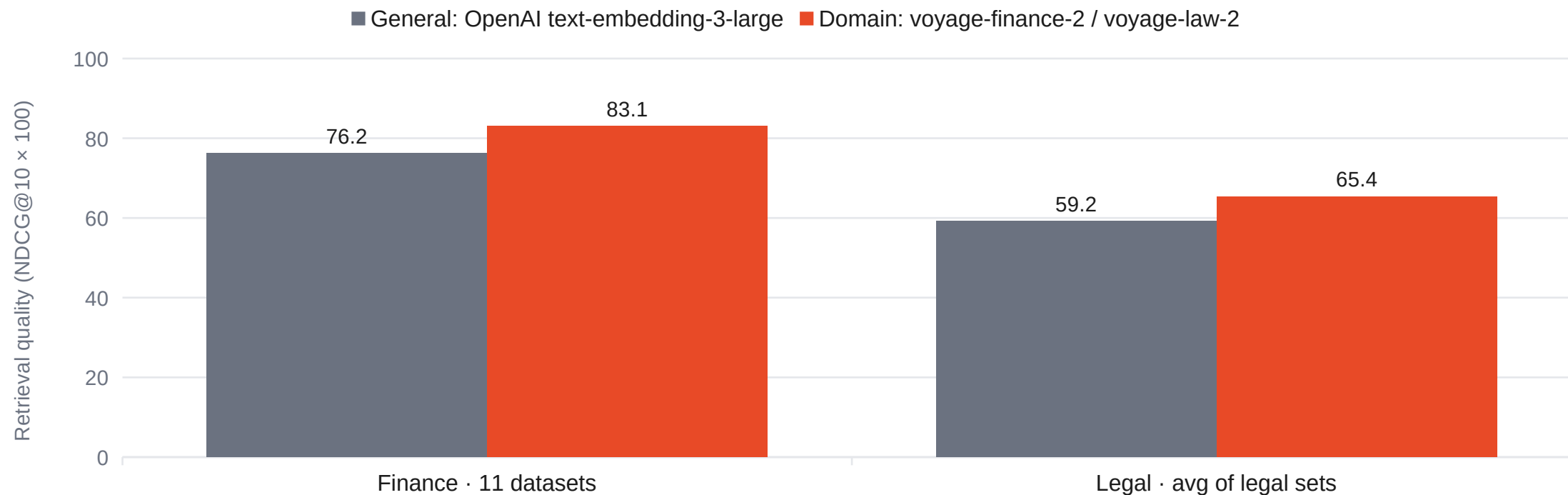
Illustrative layout: 256 numbers squeezed to 2. Closeness = cosine similarity (1 same, 0 unrelated).

Each field has its own vocabulary, so there are embedding models for each

FINANCE	HEALTHCARE	LEGAL	CODE
voyage-finance-2 Voyage AI · API · 2024 TRAINED ON Filings, corporate news, financial analysis GOOD FOR 10-Ks, earnings calls, research notes	MedCPT NCBI / NIH · open weights TRAINED ON 255M real PubMed searches and the articles people clicked GOOD FOR Searching medical literature	voyage-law-2 Voyage AI · API · 2024 TRAINED ON An extra 1T tokens of legal text GOOD FOR Case law, contracts, long legal docs	voyage-code-4 Voyage AI · API TRAINED ON Code and programming text GOOD FOR Finding functions in a codebase

Our book uses a general model: OpenAI text-embedding-3-small.

On their own field, domain models beat a top general model by 6–7 points



Higher = the right passage ranks nearer the top. Worth testing on your own documents.

Voyage AI blog: finance (Jun 2024), legal (Apr 2024). Vendor-reported.

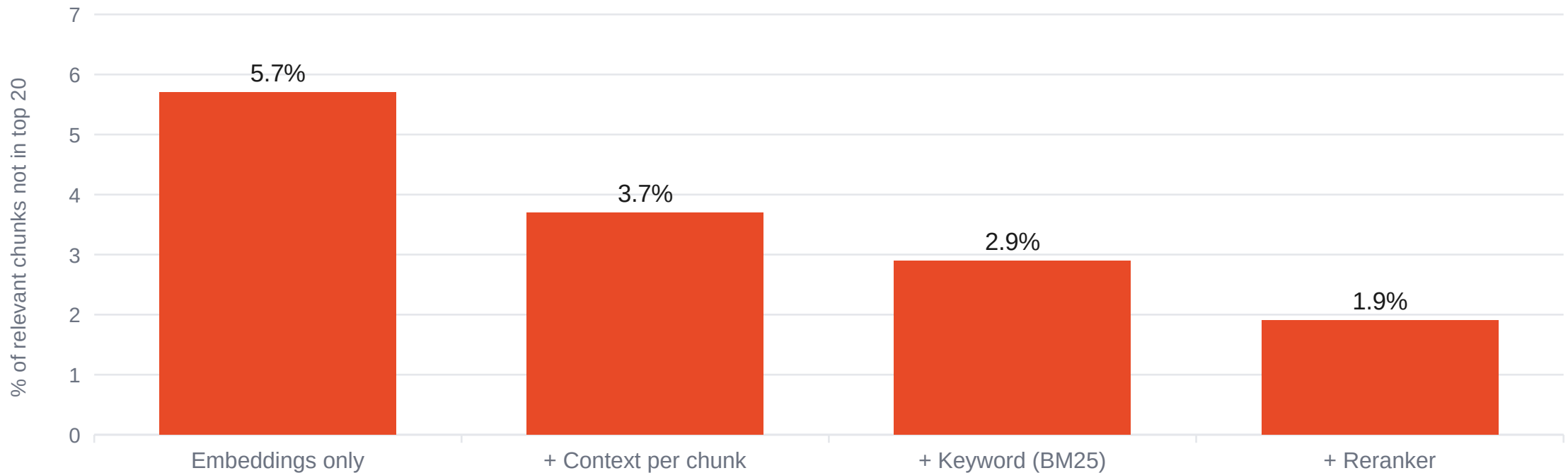
Four ways to retrieve

Each search method fixes one blind spot and has another

Method	Ranks by	Good at	Misses
Keyword (BM25)	Shared words, rare ones weigh more	Tickers, codes, exact terms	Same idea, other words
Semantic	Embedding similarity	Paraphrase	Rare exact terms
Hybrid	Merges both rankings	Most questions	When one list is badly wrong
Rerank	Rescores hybrid's top 10	Best of a good shortlist	Anything the shortlist missed

EVIDENCE

Stacking keyword search and a reranker cut missed passages by two-thirds



The steps stack: each one removes misses the last one left.

Anthropic, Introducing Contextual Retrieval (2024). Average across domains, top-20 retrieval.

CHECKPOINT

A user searches a support database for an exact order ID, like INV-58213.

Which search method is most likely to rank the right record first?

A Keyword or hybrid: an exact code is a rare word to match

B Semantic: it understands what an order is

C None; IDs cannot be searched

CHECKPOINT · ANSWER

A user searches a support database for an exact order ID, like INV-58213.

Which search method is most likely to rank the right record first?

A Keyword or hybrid: an exact code is a rare word to match

B Semantic: it understands what an order is

C None; IDs cannot be searched

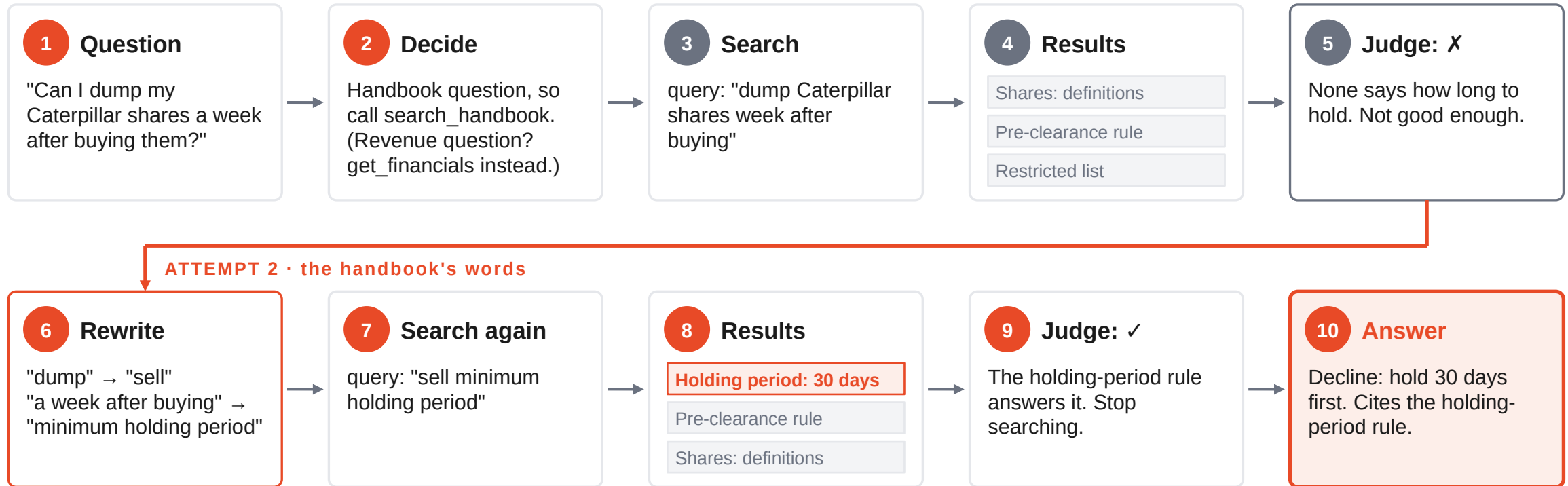
Exact codes and tickers are keyword search's home turf. Semantic blurs them into their topic.

PART 1 · RETRIEVAL AND RAG

Agentic retrieval

The agent searches, judges the results, and searches again

ATTEMPT 1 · the user's words



Same chapter 1 loop, one more tool. The step budget stops endless searching.

Worked example: pre-clearance

WORKED EXAMPLE

Four requests, four different rules the agent must find

Request	Trade	What's different	Expected
7101	Priya, buy 50 DE	Last published on Deere 41 days ago	Approve
7102	Marcus, sell 200 CAT	Position held 12 days	Decline: 30-day hold
7103	Jordan, buy 30 NVDA	NVIDIA is on the restricted list	Decline
7104	Tom, sell 80 DE	Published on Deere 6 days ago	Hold: 14-day blackout

Read-only tools. No clear_trade. The agent recommends; compliance approves.

CITATIONS

A citation only helps if someone checks it

- Real clause, says what the answer claims: approve in seconds.
- Points nowhere, or to a different rule: worse than none.
- A wrong citation looks checked.

CHECKPOINT

An AI assistant gives you an answer and cites a source document.

Before you act on the answer, what should you do?

- A Nothing; a cited answer has already been checked
- B Ask the assistant whether it is sure
- C Open the source and check it says what the answer claims

CHECKPOINT · ANSWER

An AI assistant gives you an answer and cites a source document.

Before you act on the answer, what should you do?

- A Nothing; a cited answer has already been checked
- B Ask the assistant whether it is sure
- C Open the source and check it says what the answer claims**

A citation tells you where to look. It isn't proof until a person reads it.

02

Short-term memory

Keep a long conversation going without letting it outgrow the desk.

PART 2 · SHORT-TERM MEMORY

Background

Short-term memory works like yours in a conversation



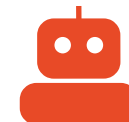
YOU

Following a dinner conversation as it happens

Holding only about 4 things in mind at once

The start of a long story slips away

Next day you recall the gist, not the words



THE AGENT

The context window: this chat, turn by turn

A desk with a fixed number of tokens

Trim: the oldest turns are dropped

Summarize: older turns become a short summary

And like you, once the conversation ends, it's gone unless someone writes it down.

Working memory holds about 4 chunks: Cowan (2001). Miller (1956) famously said 7 ± 2 .

NO MEMORY OF ITS OWN

Every model call starts with an empty desk

- The model reads only what's on the desk this call.
- "Memory" = what code puts back from before.
- Next Monday, "And NVIDIA?" means nothing.

TWO KINDS

Short-term and long-term memory solve different problems

	Short-term	Long-term
Holds	This conversation, turn by turn	Facts worth keeping after it ends
Lasts	Until the session ends	Weeks, months
Problem	Outgrows the desk	What to keep, and keeping it right
At the firm	Today's back-and-forth with Dana	Dana prefers tables

PART 2 · SHORT-TERM MEMORY

Trim or summarize

The desk is full. Trim drops the start; summarize shrinks it

The chat so far

4 turns · desk is full

Compare Deere and Caterpillar revenue growth last quarter.

Deere +6.4% to \$13.8B. Caterpillar +3.1% to \$16.9B.

I prefer tables. Keep it under 150 words.

Understood.

Trim: keep last 2

cheap; dashed turns are dropped

Compare Deere and Caterpillar revenue growth last quarter.

Deere +6.4% to \$13.8B. Caterpillar +3.1% to \$16.9B.

I prefer tables. Keep it under 150 words.

Understood.

And NVIDIA?

✗ "NVIDIA compared to what?"

Summarize, keep last 2

costs a model call

SUMMARY Compared Deere (+6.4%, \$13.8B) and Caterpillar (+3.1%, \$16.9B).

I prefer tables. Keep it under 150 words.

Understood.

And NVIDIA?

✓ **NVIDIA added to the table**

Pick by asking: will something said early matter later?

Trim

each turn stands on its own

IT help desk bot

Password resets, VPN fixes: last few turns are all that matter

Order-status chat

Each question looks the order up fresh

Live price checker

"What's NVDA at?" Data is fetched again every time

Summarize

early details shape later answers

Wealth-advisor assistant

Risk tolerance said in minute 1 still matters in minute 40

Support escalation

Hand the human agent a summary, not 60 messages

Research session

An afternoon on one company: keep figures and decisions

Pin it

must never be dropped

Client restrictions

"No tobacco stocks" stays on the desk every turn

Compliance facts

Account type, restricted list, blackout dates

Identity

Which client, which account, which mandate

No → trim. Yes → summarize. Must never be lost → pin it in the instructions or save it to long-term memory.

CHECKPOINT

In a long chat, a user stated a preference in their very first message.

The app keeps only the last six messages. What happens to the preference?

- A** Kept: preferences are always saved
- B** Dropped, with everything else from the early messages
- C** Moves to long-term memory automatically

CHECKPOINT · ANSWER

In a long chat, a user stated a preference in their very first message.

The app keeps only the last six messages. What happens to the preference?

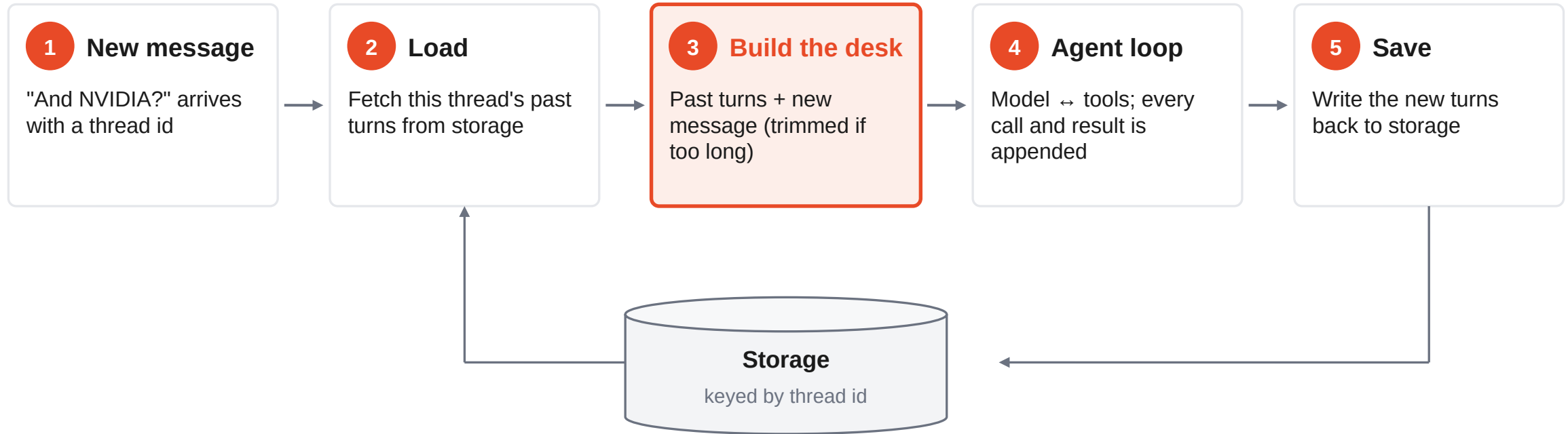
- A Kept: preferences are always saved
- B Dropped, with everything else from the early messages**
- C Moves to long-term memory automatically

Trimming doesn't know what matters. Nothing is 'automatic' unless code does it.

PART 2 · SHORT-TERM MEMORY

Inside the frameworks

Every turn: load the thread, run the loop, save what's new



The model still keeps nothing. The framework does the remembering.

Agents SDK: a session replays the conversation before every run

PYTHON

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(name="Analyst")
session = SQLiteSession("dana-monday", "memory.db")

# turn 1: history empty, turn gets saved
await Runner.run(agent, "Compare DE vs CAT",
                 session=session)

# turn 2: turn 1 is loaded first
await Runner.run(agent, "And NVIDIA?",
                 session=session)
```

1

Before the run

`get_items()`: load past turns, put them first

2

During the run

model + tools, as in chapter 1

3

After the run

`add_items()`: save input, replies, tool calls

4

Too long?

`SessionSettings(limit=50)`: load only the last 50

WHERE IT'S STORED

SQLite

Redis

SQLAlchemy

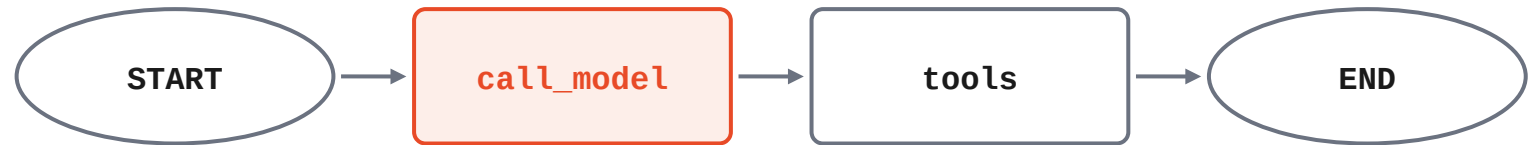
MongoDB

Encrypted

In LangGraph, short-term memory is the state, saved every step

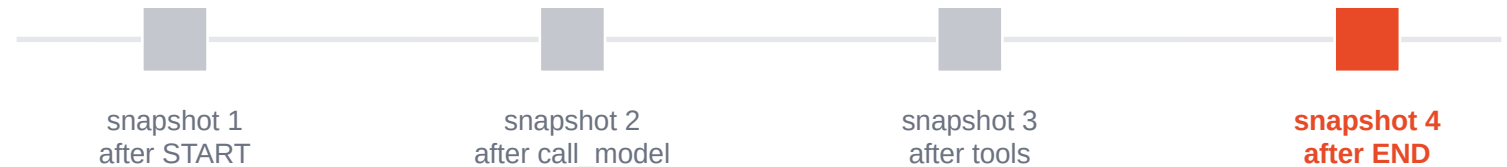
STATE

```
messages: [  
  human: Compare DE vs CAT  
  ai: DE +6.4%, CAT +3.1%  
  human: I prefer tables  
  human: And NVIDIA?  
]  
client: "meridian"
```



each node reads the state and returns updates; add_messages appends

CHECKPOINTER · thread_id = "dana-monday"



Same thread_id next call → **the latest snapshot is loaded and the conversation continues.** New thread_id → empty state.

LangGraph: compile with a checkpointer, call with a thread_id

PYTHON

```
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.memory import InMemorySaver

def call_model(state: MessagesState):
    return {"messages": model.invoke(state["messages"])}

builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "dana-monday"}}
graph.invoke({"messages": [("user", "And NVIDIA?")]}), cfg)
```

1

Trim

trim_messages(): keep the last N tokens on the desk

2

Delete

RemoveMessage(id): drop turns from the state

3

Summarize

SummarizationNode: fold old turns into a summary

4

Inspect

graph.get_state(cfg): see exactly what's remembered

PRODUCTION CHECKPOINTERS

PostgresSaver

RedisSaver

MongoDBSaver

SqliteSaver

03

Long-term memory

Decide what to save between sessions, who writes it, and how to catch a wrong note.

Long-term memory comes in the same three kinds as yours



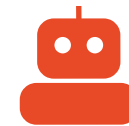
YOU

Episodic: your first day of the MSBA

Semantic: your best friend is vegetarian

Procedural: riding a bike, your morning routine

Writing it in your notes app or calendar



THE AGENT

What we told Dana on Monday, and when

Dana prefers tables, under 150 words

How the firm writes a comparison memo

The harness or agent saving notes

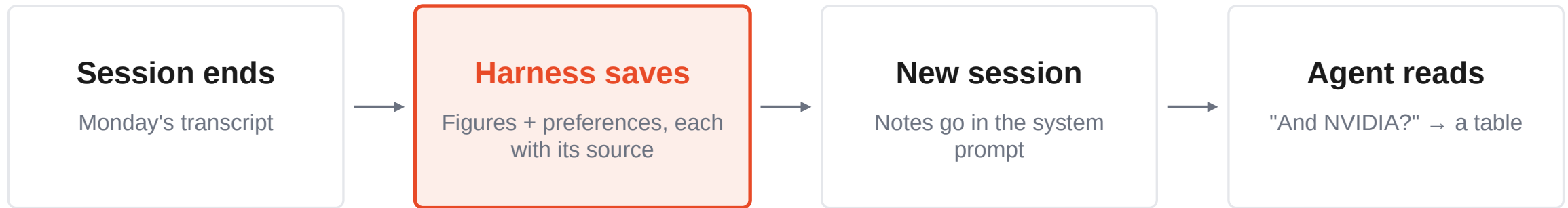
You don't replay every conversation. You keep what matters, and sometimes remember it wrong.

Episodic vs semantic memory: Endel Tulving (1972).

PART 3 · LONG-TERM MEMORY

Notes written by the harness

The harness saves notes when a session ends and recalls them next time



The agent only ever reads the notes. The rules decide what's kept.

Three kinds of memory

Each kind of memory is written, found and fixed differently

	Episodic	Semantic	Procedural
It is	What happened, when	What's true about the client	How the firm does the task
Example	Memo told Dana DE +6.4%	Dana prefers tables	Comparisons: both, cite each
Found by	Recency, similarity	Client's name	The task at hand
Fixed by	Re-check old figures	Replace when it changes	A person edits it

CHECKPOINT

An assistant has a saved note: "User prefers tables." Today the user says they now want bullet points.

What should happen to the old note?

- A** Keep it, and add a new note beside it
- B** Replace it: it's a standing fact that is now wrong
- C** Leave it; the assistant will work out which is newer

CHECKPOINT · ANSWER

An assistant has a saved note: "User prefers tables." Today the user says they now want bullet points.

What should happen to the old note?

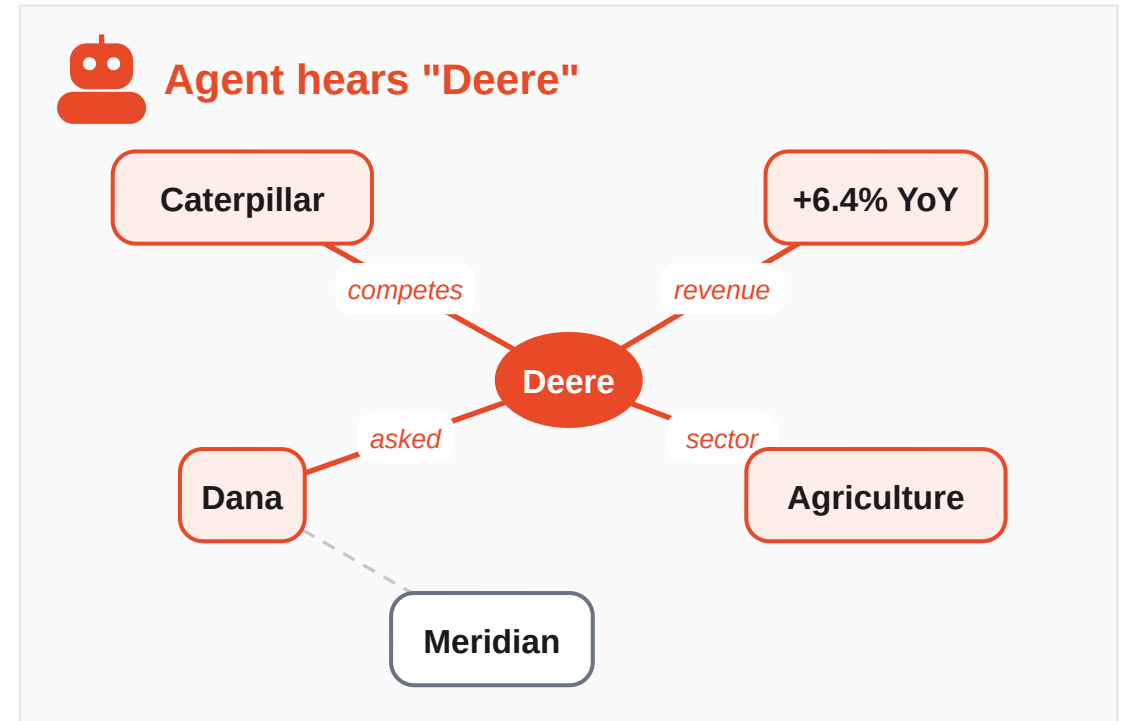
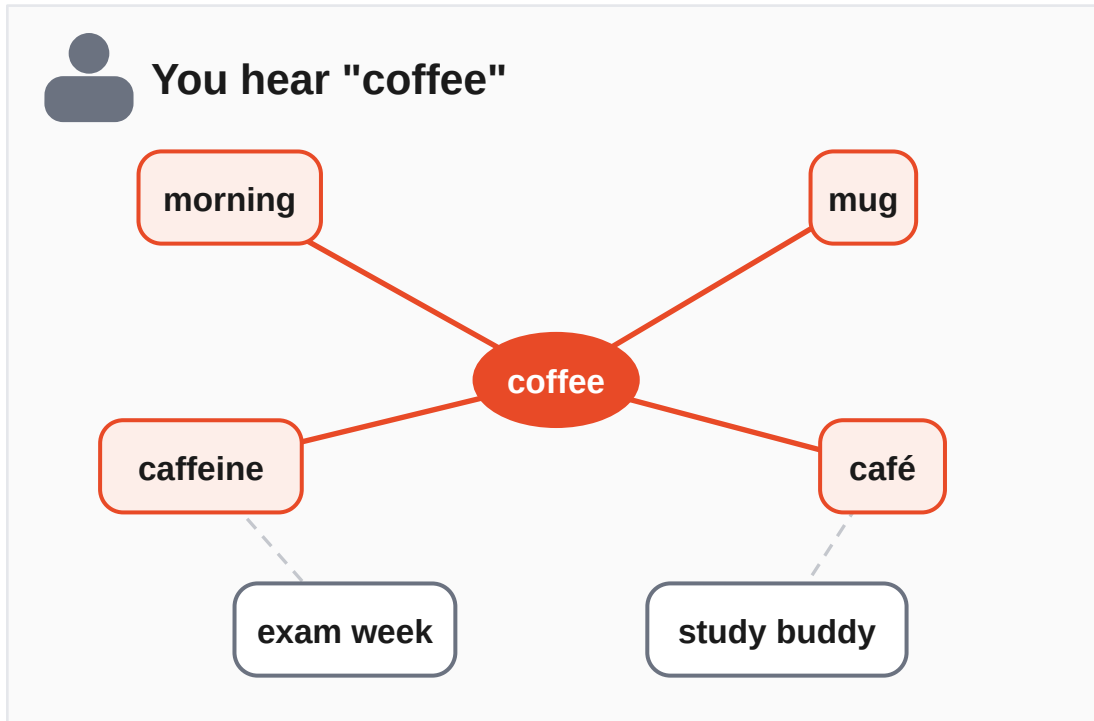
- A Keep it, and add a new note beside it
- B Replace it: it's a standing fact that is now wrong**
- C Leave it; the assistant will work out which is newer

Facts about a person get replaced, not stacked. Two contradicting notes confuse the agent.

PART 3 · LONG-TERM MEMORY

Graph memory

Your memory is a web: one idea lights up the ones linked to it



Orange = one hop, lit first. Dashed = two hops, lit if needed.

Psychology: Collins & Loftus (1975), spreading activation in semantic memory. Agent side: illustrative.

A graph is just things (nodes) joined by named links (edges)



Entity

A real-world thing: a person, company, product, place

Node

How an entity is stored in the graph, with properties

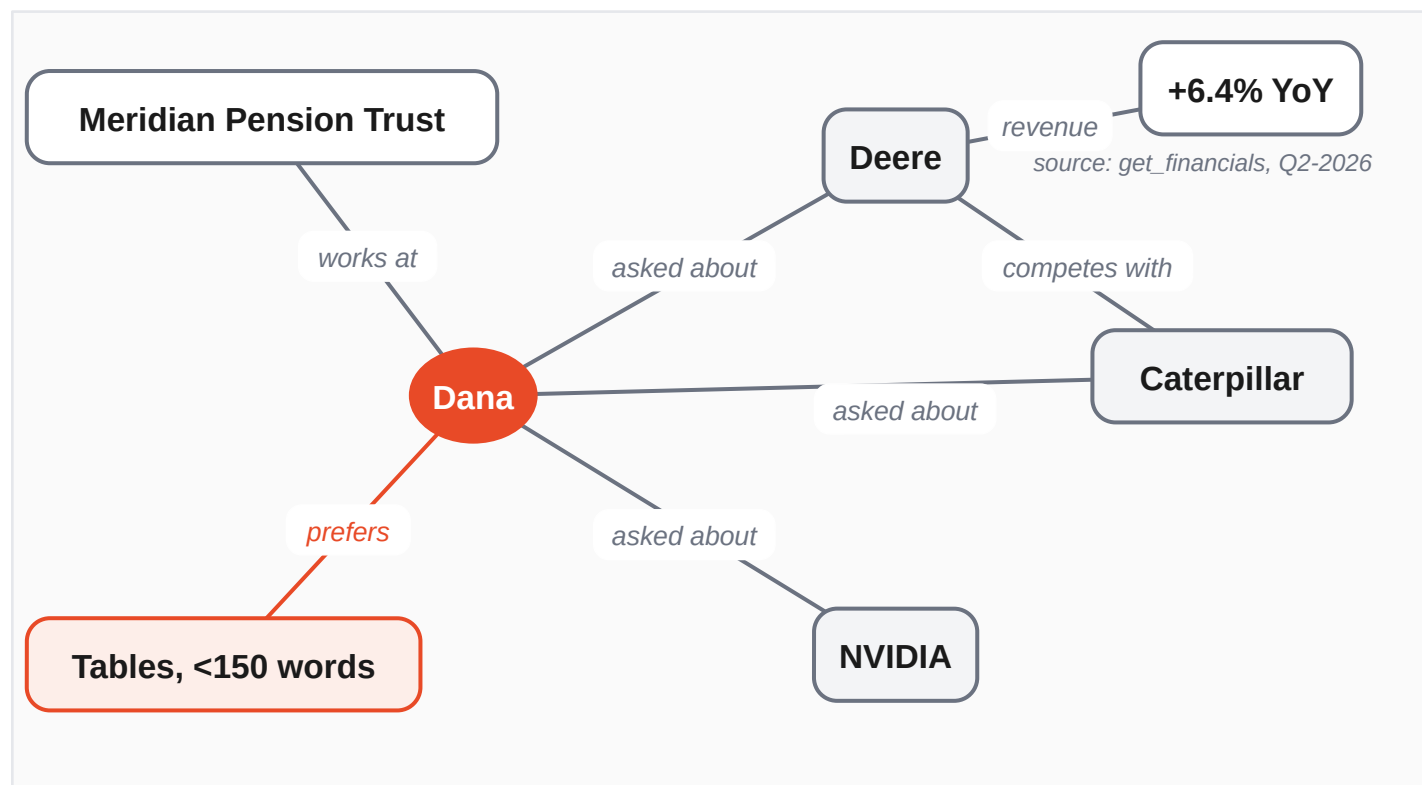
Edge

A named, one-way link between two nodes

Triple

(Dana) → asked about → (Deere): one fact

Graph memory stores facts as things and the links between them



1

Extract

An LLM turns each chat into triples: (Dana, prefers, tables)

2

Store

Things become nodes, links become edges. Nodes also get embeddings

3

Retrieve

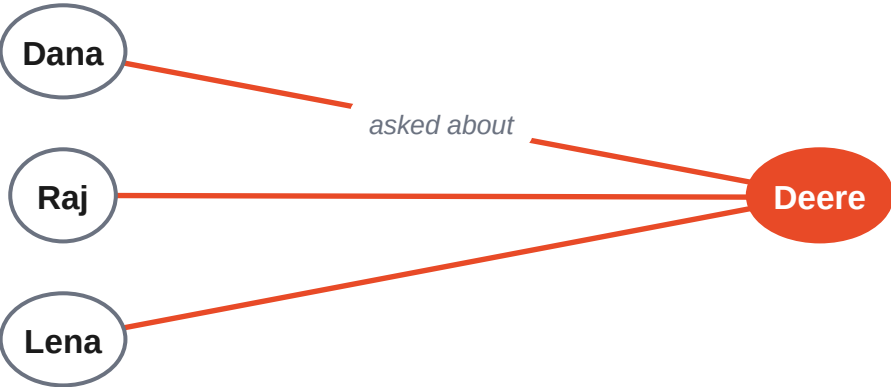
Find the matching node, walk 1–2 links, put that piece on the desk

The meaning lives in the links, not just in the words.

Graphs answer relationship questions and keep a fact's history

"Which clients asked about Deere?"

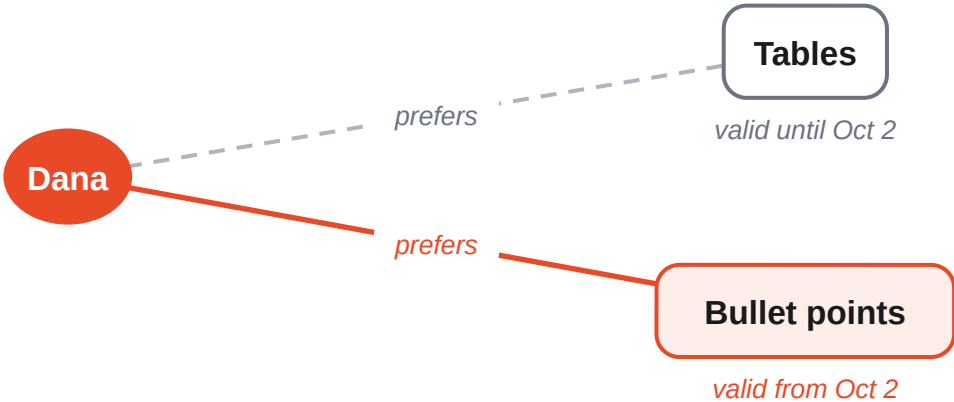
Flat notes search every note's text and hope the wording matches



Graph start at Deere, follow "asked about" back

IN THE WILD Microsoft GraphRAG (2024) · Zep / Graphiti (2025) · Mem0 graph memory (2025) · Neo4j graph database

"Dana now wants bullet points"



Old edge is closed, not deleted, so history survives.

PART 3 · LONG-TERM MEMORY

Notes written by the agent

Letting the agent write notes catches more, mistakes included

Harness writes

Saves only what its rules target

Predictable

Changing it means editing code

Agent writes

remember_note tool: it chooses

Catches what no rule anticipated

Can save its own mistakes

First time this series an agent writes: only to its own notebook, visible to a person.

When memory goes wrong

A REMEMBERED FIGURE

6.4% → 64.0%

The agent saved Deere's growth wrong. Next week's memo cites get_financials anyway.

Scripted in the textbook's mock model. A real model slips only sometimes: harder to catch.

THREE GUARDS

Three guards stop a wrong note reaching a client

- Keep the source with every note. (records-4)
- Re-check a remembered figure before it goes out. (records-5)
- Let a person see and delete notes. (records-4)

A note is a reminder of where to look, not evidence.

CHECKPOINT

An assistant's memory says a company's revenue grew 64%. The real figure was 6.4%.

Which safeguard stops the wrong number reaching a client report?

- A Re-checking every remembered figure against the source before it's used
- B Telling the assistant to be more careful in its instructions
- C Keeping memories for a shorter time

CHECKPOINT · ANSWER

An assistant's memory says a company's revenue grew 64%. The real figure was 6.4%.

Which safeguard stops the wrong number reaching a client report?

A Re-checking every remembered figure against the source before it's used

B Telling the assistant to be more careful in its instructions

C Keeping memories for a shorter time

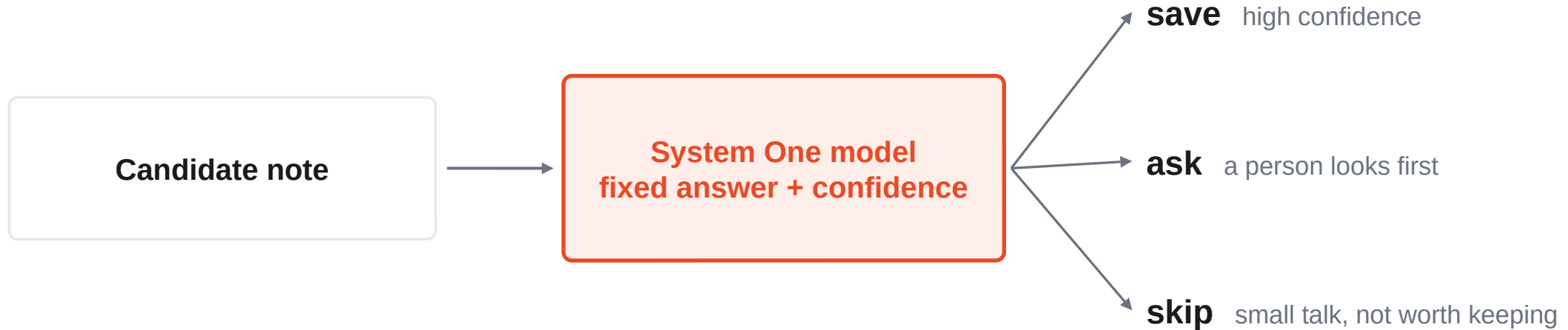
Only a check against the source catches a wrong number. Instructions don't.

PART 3 · LONG-TERM MEMORY

A fast gate

A SYSTEM ONE GATE

A typed gate decides each note: save, ask a person, or skip



It can't answer with a paragraph, or invent a fourth option.

WORKED EXAMPLE

Four candidate notes; only the doubtful one reaches a person

- 1 "Dana wants tables, under 150 words" → save
- 2 "Deere grew 64.0%", source: agent's summary → ask
- 3 "Thanks, that's all for today" → skip
- 4 "Deere grew 6.4%", source: get_financials → save

Three decided automatically. The risky one waits for a human.

Memory in real frameworks

Real frameworks draw the same line between short- and long-term

	Short-term	Long-term
OpenAI Agents SDK	Session: SQLiteSession stores every turn	Not built in: add a remember tool
LangGraph	Checkpointers, keyed by thread_id	Store: notes in namespaces, with search

Frameworks add persistence and scale. Judgment about what to keep is still yours.

EXERCISE · 3.1

Retrieval: break it, then explain why

- Find a question where hybrid beats semantic. And one the reranker flips.
- Re-chunk: 40 words, 10 overlap. Which questions got worse?
- Check the cited clause id exists and shares a key term.

One sentence of "why" for each answer.

Memory: remove it, inspect it, decide what to keep

- Agents SDK: drop the remember tool. What survives next session?
- LangGraph: make every note carry a source, or refuse it.
- Three sentences: agent may keep, harness only, never keep.

Popular AI assistants all use the memory ideas from today

Product	What it remembers	Today's term	Your controls
ChatGPT (OpenAI)	Saved memories + relevant bits of past chats	Agent-written notes + retrieval	View, edit, delete; Temporary Chat
Claude (Anthropic)	Work context, kept separately per project	Notes scoped by project	Editable summary; Incognito chats
Gemini (Google)	Personal Context from past chats	Retrieval over past sessions	Toggle off; Temporary Chats, 72 hrs
Microsoft Copilot	Preferences, interests, milestones	Semantic memory	Dashboard to manage or delete
Claude Code	CLAUDE.md you write + auto memory it writes	Procedural + agent notes	Plain files you can edit

Every one gives you the same three controls: see it, fix it, switch it off.

Where RAG and memory actually live: databases and frameworks

Vector databases

store embeddings for RAG search

Pinecone

managed, serverless · www.pinecone.io

Chroma

open source, easy start · www.trychroma.com

Weaviate

open source, hybrid search · weaviate.io

Qdrant

open source, fast filters · qdrant.tech

pgvector

vectors inside Postgres · github.com/pgvector/pgvector

FAISS

Meta's search library · github.com/facebookresearch/faiss

Graph databases

store nodes and edges

Neo4j

most widely used · neo4j.com

Amazon Neptune

managed on AWS · aws.amazon.com/neptune

Memgraph

in-memory, fast · memgraph.com

FalkorDB

built for GraphRAG · www.falkordb.com

ArangoDB

graph + document · arangodb.com

Graph RAG & memory

frameworks on top

Microsoft GraphRAG

graph from documents · github.com/microsoft/graphrag

Graphiti (Zep)

time-aware agent memory · github.com/getzep/graphiti

Mem0

memory layer, graph option · github.com/mem0ai/mem0

LlamaIndex

RAG pipelines · www.llamaindex.ai

LangGraph memory

state + store · docs.langchain.com

What to take away

- 01 RAG: retrieve, put on the desk, answer from it, cite the clause.
- 02 Hybrid + rerank, and hand the model a few candidates.
- 03 Short-term memory: trim or summarize; both lose something.
- 04 Long-term: harness or agent writes; episodic, semantic, procedural.
- 05 A remembered figure looks checked. Keep sources; re-check.

BEFORE YOU GO

Scan to share your feedback



Feedback · Workshop 3

Fri, Oct 2, 2026 · 1:30–3:30 pm · 2043 BIF

- 1 Point your phone camera at the code.
- 2 Fill in the short form.
- 3 One thing that worked, one to improve.

Questions?

GO DEEPER

Lewis et al., Retrieval-Augmented Generation (2020)

Anthropic, Introducing Contextual Retrieval (2024)

Packer et al., MemGPT (2023)

Lilian Weng, LLM Powered Autonomous Agents (2023)

OpenAI Agents SDK: Sessions · LangGraph: Memory overview



Chapter 3 · textbook

Next session: Fri, Oct 16 · ML Foundations I